Applied ML & Statistics Fundamentals

A Complete Beginner-to-Advanced Study Guide: The Classical Foundations Behind Every Model

July 2026

Contents

1 How to Use This Guide

Everything exciting in AI today sits on a foundation of classical machine learning and statistics — and that foundation is exactly what most LLM-era developers skipped. This guide builds it properly: statistics literacy first (because ML *is* statistics with better marketing), then the core ML workflow, the essential algorithms, and how to evaluate models honestly. No prior math beyond school algebra is assumed; every formula is explained in words first.

The one-sentence summary: *Machine learning means fitting a function to past data so it makes good predictions on future data — and the entire craft lies in "future": measuring honestly on data the model never saw, and resisting the hundred ways models (and analysts) fool themselves.*

2 Plain-English Glossary (Read This First)

- **Feature:** An input variable the model uses (age, price, word count). A row of features describes one example.
- **Label / target:** The output you want to predict (spam or not, house price).
- **Model:** A function with adjustable knobs (*parameters*) that maps features to a prediction.
- **Training:** Adjusting the knobs to fit past data.
- **Supervised learning:** Learning from labeled examples (features → known answers).
- **Unsupervised learning:** Finding structure in unlabeled data (e.g., clustering).
- **Regression:** Predicting a number. **Classification:** Predicting a category.
- **Loss function:** A score of how wrong the model currently is; training minimizes it.
- **Gradient descent:** The knob-turning algorithm — repeatedly nudge each parameter downhill on the loss.
- **Overfitting:** Memorizing the training data (including its noise) instead of learning the pattern; great on training data, bad on new data.
- **Underfitting:** The model is too simple to capture the pattern at all.
- **Generalization:** Performing well on *new* data — the only thing that ultimately matters.
- **Train/validation/test split:** Portions of data for fitting, tuning, and the final honest exam.
- **Cross-validation:** Rotating which slice of the data is held out, to measure more reliably.
- **Hyperparameter:** A setting you choose (tree depth, learning rate) rather than a knob training learns.
- **Distribution:** The shape of how values spread (many mid-range, few extreme, etc.).
- **Mean / median / standard deviation:** Average / middle value / typical spread around the mean.
- **Correlation:** How strongly two variables move together (from -1 to +1). *Not causation.*

- **p-value:** If there were truly no effect, the probability of seeing data at least this extreme by luck.
- **Confidence interval:** A range that plausibly contains the true value, given your sample.
- **Baseline:** The dumb reference model (predict the average; predict the majority class) any real model must beat.

3 Part 1: Statistics Literacy — The Bedrock

3.1 1.1 Describing data

Before modeling anything, understand its shape:

- **Mean** (average) is intuitive but dragged around by outliers. **Median** (the middle value) is robust: median household income tells you about the typical family; the mean is inflated by billionaires. Rule: skewed data (income, latency, prices) → report the median and **percentiles** (p50, p95, p99).
- **Standard deviation (SD)** measures typical spread around the mean. For bell-shaped data, ~68% of values fall within 1 SD, ~95% within 2 SD.
- **Always plot first.** Anscombe's famous quartet: four datasets with identical means, variances, and correlations — one is a clean line, one a curve, one has a single outlier ruining everything. Summary numbers hide; histograms and scatter plots reveal.

3.2 1.2 Probability distributions you'll actually meet

- **Normal (bell curve):** heights, measurement noise, averages of many small effects (the Central Limit Theorem says sums/averages tend toward normal — the reason the bell curve is everywhere).
- **Skewed / long-tailed:** income, city sizes, web latency, virality. Means mislead; percentiles rule. Never assume normality for these.
- **Binomial / Bernoulli:** counts of yes/no outcomes (conversions out of visitors).
- **Poisson:** counts of events per interval (support tickets per hour).

3.3 1.3 Sampling: the source of all uncertainty

You almost never see the whole population — you see a sample, and samples wobble. Key intuitions:

- **Standard error:** the wobble of your *estimate* shrinks with the square root of sample size. To halve the noise, you need 4x the data. (This is why A/B tests need surprisingly many users.)
- **Confidence interval (CI):** "conversion rate 4.2%, 95% CI [3.6%, 4.8%]" — a range that would capture the true rate in ~95% of repeated experiments. Wide CI = you basically don't know yet.
- **Bias beats noise as a danger:** a big sample of the *wrong people* (survivorship bias, self-selection, only-power-users) is confidently wrong. The classic: WWII

engineers reinforced planes where returning planes showed bullet holes — until Abraham Wald pointed out those were the survivable hits; reinforce where returning planes show *no* holes, because planes hit there never returned.

3.4 1.4 Hypothesis testing and p-values (without the cargo cult)

The logic of an A/B test: assume nothing changed (the *null hypothesis*), then ask how surprising your observed difference would be under that assumption. That surprise level is the **p-value**. Small p (conventionally < 0.05) → the "pure luck" explanation looks weak.

What people get wrong, constantly:

- p is **not** "the probability the null is true," and 1 - p is **not** "the probability your change works."
- **Statistical vs. practical significance:** with 10 million users, a +0.01% lift can be "significant" and still worthless. Always report the *effect size* with its CI, not just the verdict.
- **Multiple comparisons:** test 20 metrics and one will hit p < 0.05 by luck alone. Decide the primary metric *before* the experiment.
- **Peeking:** checking the test daily and stopping the moment it "wins" massively inflates false positives. Fix the sample size (power analysis) in advance, or use methods built for sequential looks.
- **Power:** an underpowered test that "found nothing" mostly proves you didn't collect enough data — absence of evidence is not evidence of absence.

3.5 1.5 Correlation, causation, and the classic traps

- **Correlation is symmetric and dumb:** ice cream sales correlate with drowning (both follow summer). The hidden common cause is a **confounder**. Only randomized experiments (or careful causal methods) separate causation from correlation.
- **Simpson's paradox:** a trend can hold in every subgroup yet reverse in the aggregate (a treatment can look worse overall simply because it was given to sicker patients). Moral: check subgroups; know how the data was generated.
- **Regression to the mean:** extreme measurements are usually part luck; the follow-up measurement drifts back toward average *on its own*. This masquerades as "my intervention worked" after any bad streak.
- **Base rates:** a 99%-accurate test for a 1-in-10,000 condition still yields mostly false positives — because the disease is so rare, false alarms vastly outnumber true hits. (This is Bayes' theorem in action, and exactly why rare-event classifiers need more than "accuracy.")

4 Part 2: The Machine Learning Workflow

4.1 2.1 What learning actually is

A model is a function `prediction = f(features; parameters)`. Training = choose parameters minimizing a **loss** over training examples (mean squared error for regression; cross-entropy/log loss for classification), usually via **gradient descent**: compute the slope of the loss with respect to each parameter, step downhill, repeat.

Analogy: you're blindfolded on a hillside trying to reach the valley. You feel the slope under your feet (the gradient) and step downhill. Step size = the *learning rate*: too small and you'll walk forever; too big and you'll overshoot back and forth across the valley.

4.2 2.2 The central drama: bias vs. variance

- **Underfitting (high bias):** the model is too simple — a straight line through a curve. Bad on training data *and* new data.
- **Overfitting (high variance):** the model is too flexible — it threads through every training point, noise included. Superb on training data, poor on new data.

Analogy: a student who memorizes past exam papers word-for-word (overfit) aces the mock and bombs the real exam; a student who skimmed one summary page (underfit) fails both; the student who learned the *concepts* generalizes.

Diagnosis is mechanical: **training error vs. validation error.** Both high → underfitting (add features, use a more flexible model). Training low but validation much higher → overfitting (more data, simpler model, regularization).

Regularization = deliberately penalizing complexity: L2/ridge shrinks all weights toward zero; L1/lasso pushes some weights *exactly* to zero (automatic feature selection); trees get depth limits and pruning; neural nets get dropout and early stopping.

4.3 2.3 The sacred split (where most self-deception happens)

- **Training set** — fit parameters.
- **Validation set** — compare models and tune hyperparameters. (Because you tune *against* it, your final validation score is slightly flattering.)
- **Test set** — touched **once**, at the very end. The honest exam.

Cross-validation (k-fold): split data into k parts; train on k-1, validate on the held-out part; rotate; average. More reliable than a single split, at k-times the compute — the default for small/medium datasets.

The cardinal sin — data leakage: information from the future or from the answer sneaking into the features or preprocessing. Classics:

- Scaling/normalizing using statistics computed on *all* data before splitting (the test set leaks into training). Fit preprocessing on training data only.
- Random splits on **time-series** data — the model trains on Tuesday to "predict" Monday. Always split chronologically for temporal data.
- A feature that is a proxy for the label ("account_closed_date" when predicting churn).

- Duplicate/near-duplicate rows landing on both sides of the split.

Leakage is why "99% accurate" prototypes collapse in production. If a result looks too good, hunt for leakage before celebrating.

4.4 2.4 An honest end-to-end recipe

1. Define the prediction task and the *business* metric it serves.
2. Establish a **baseline** (predict the mean/majority; or a one-rule heuristic). Every model must beat it to justify existing.
3. Explore and plot the data; handle missing values and outliers *deliberately* (and identically in production).
4. Split properly (chronologically if temporal). Freeze the test set.
5. Start simple (linear/logistic regression). Then try gradient-boosted trees. Escalate only if justified.
6. Tune hyperparameters on validation/CV. Inspect errors by hand — error analysis finds more improvement than tuning does.
7. One final test-set evaluation. Report metric *with uncertainty*.
8. In production: monitor input **drift** (feature distributions shifting) and performance decay; retrain on a schedule or trigger.

5 Part 3: The Essential Algorithms (What, How, When)

5.1 3.1 Linear regression — the fruit fly of ML

Predicts a number as a weighted sum: `price = w1*area + w2*rooms + b`. Trained by minimizing squared error. Why it's still everywhere: instantly trained, interpretable (each weight = the effect of one unit of that feature, other things equal), a strong baseline, and the conceptual ancestor of neural networks (a neural net is stacked linear regressions with nonlinear squashes between them). Caveats: assumes roughly linear effects; sensitive to outliers (squared error explodes); correlated features make individual weights untrustworthy (multicollinearity).

5.2 3.2 Logistic regression — classification's workhorse

Linear regression squeezed through a sigmoid so the output is a probability between 0 and 1: `P(churn) = sigmoid(w.x + b)`. Trained with log loss. Despite the name, it's a classifier — and an astonishingly hard baseline to beat on wide, sparse data (text bag-of-words, click prediction). Bonus: well-calibrated probabilities and readable coefficients.

5.3 3.3 Decision trees — human-readable rules

A tree of if/else splits chosen to maximize purity of the resulting groups ("income > 50k? → yes branch…; age < 30? → …"). Pros: no scaling needed, handles nonlinearity and interactions natively, fully explainable. Con: a single deep tree overfits wildly — it will happily grow a leaf per training example. Which is why almost nobody ships one tree; they ship forests.

5.4 3.4 Ensembles — where tabular ML actually lives

Two ways to combine many weak trees into one strong model:

- **Random Forest (bagging):** train hundreds of trees, each on a bootstrap sample of rows and a random subset of features; average their votes. Errors of diverse trees cancel out — wisdom of a decorrelated crowd. Nearly tuning-free, robust, a superb first serious model.
- **Gradient Boosting (XGBoost / LightGBM / CatBoost):** train trees *sequentially*, each new tree fitting the errors of the ensemble so far. State of the art on tabular data for over a decade — the honest answer to "what wins on spreadsheet-shaped data" is still *boosted trees, not deep learning*. Needs more tuning (tree depth, learning rate, number of trees) and can overfit if pushed.

5.5 3.5 k-Nearest Neighbors and k-Means (and their descendants)

- **kNN (supervised):** predict by looking at the k most similar training examples and voting. No training phase; all cost at prediction time; degrades in high dimensions. Conceptual note: *vector search in RAG (folder 02) is literally kNN over embeddings.*
- **k-Means (unsupervised clustering):** pick k centers, assign points to the nearest center, move centers to their cluster's mean, repeat. Used for segmentation, anomaly context, data exploration. You must choose k (elbow/silhouette heuristics) and it prefers blob-shaped clusters.
- **PCA (dimensionality reduction):** rotate the data to find the few directions holding most variance; compress hundreds of correlated features into a handful of components for visualization or speed.

5.6 3.6 Neural networks — one honest paragraph

Layers of linear transformations with nonlinear activations between them, trained end-to-end by gradient descent (backpropagation = the chain rule applied efficiently). They shine when features must be *learned* from raw signal — images, audio, and language (LLMs are enormous neural networks over token sequences). On modest-sized tabular business data they usually lose to boosted trees while costing more. Everything in this guide — loss functions, overfitting, regularization, splits, leakage, evaluation — applies to them unchanged; deep learning is classical ML with billions of knobs.

6 Part 4: Evaluating Models Honestly

6.1 4.1 Regression metrics

- **MAE** (mean absolute error): average miss, in the target's own units. Robust, interpretable.
- **RMSE:** like MAE but squares errors first, so big misses are punished extra. Compare to a baseline's RMSE, always.

- **R^2:** fraction of the target's variance the model explains (1 = perfect, 0 = no better than predicting the mean, negative = worse than the mean).

6.2 4.2 Classification metrics — the accuracy trap and the confusion matrix

The trap: 99.7% of transactions are legitimate → a model that says "legit" for everything is 99.7% *accurate* and 100% useless. With imbalanced classes, accuracy lies. The cure is the **confusion matrix**:

```
                    predicted fraud    predicted legit
actual fraud           TP  90              FN  10      (missed fraud)
actual legit           FP  400             TN  99,500  (false alarms)
```

- **Precision** = TP / (TP+FP): of the alarms raised, how many were real? (90/490 = 18%)
- **Recall** = TP / (TP+FN): of the real fraud, how much did we catch? (90/100 = 90%)
- **F1** = harmonic mean of the two — a single number when both matter.
- **The trade-off is a dial, not a fact:** the model outputs probabilities; *you* pick the threshold. Lower threshold → more recall, worse precision. Choose it from the real costs: spam filters want precision (never eat a real email); cancer screening wants recall (never miss a case; false alarms get a second test).
- **ROC-AUC:** threshold-free ranking quality — probability a random positive scores above a random negative (0.5 = coin flip, 1.0 = perfect). With heavy imbalance prefer **precision-recall curves/AUC-PR**, which don't get flattered by the ocean of easy negatives.
- **Calibration:** does "70% confident" come true 70% of the time? Crucial whenever probabilities feed decisions or prices.

6.3 4.3 Beyond one number

- Report uncertainty (CV spread, bootstrap CIs) — a 0.3% metric gap between models is usually noise.
- **Slice the metrics:** overall numbers hide segment failures (new users, one country, one device). Evaluate per segment — this is also where fairness problems surface.
- **Error analysis:** read 50 wrong predictions with your own eyes. Nothing else generates better ideas for features, data fixes, and product decisions.

7 Part 5: Common Mistakes, Best Practices, FAQs

7.1 Common mistakes

1. **No baseline** — celebrating 92% accuracy when "always predict majority" scores 91%.
2. **Data leakage** — the too-good-to-be-true score (preprocess-before-split, random splits on time data, proxy features, duplicates).

3. **Accuracy on imbalanced data** — use the confusion matrix, precision/recall, AUC-PR.
4. **Touching the test set repeatedly** — it silently becomes a validation set; the final number stops being honest.
5. **p-hacking analytics** — many metrics, peeking, post-hoc subgroup fishing.
6. **Confusing correlation with causation** — shipping decisions on confounded observational data.
7. **Ignoring effect sizes and CIs** — "significant" ≠ "matters."
8. **Deep learning by default on tabular data** — boosted trees are the proven default there.
9. **Train-serve skew** — production preprocessing differing from training preprocessing.
10. **Set-and-forget deployment** — the world drifts; monitor and retrain.

7.2 Best practices

- Plot before modeling; medians/percentiles for skewed data.
- Baseline → linear/logistic → boosted trees → (only if justified) deep learning.
- Freeze the test set; cross-validate; fit all preprocessing on training folds only; split time data chronologically.
- Pick the metric from the cost of each error type; set thresholds from business trade-offs.
- Pre-register your A/B test's primary metric and sample size; report effect sizes with CIs.
- Do manual error analysis every iteration; slice metrics by segment.
- Version data + code + model together; monitor drift in production.

7.3 FAQs

Q: How much math do I need? A: For applied work: comfort with means/variance/percentiles, probability basics, reading a formula, and the intuitions in Part 1. Calculus/linear algebra deepen understanding (gradients, PCA) but libraries handle the mechanics.

Q: How much data do I need? A: Depends on signal strength and model complexity. Hundreds of rows can support linear models; boosted trees like thousands+; deep nets want far more. Learning curves (performance vs. training size) answer it empirically for *your* problem.

Q: My model is 95% accurate — good? A: Compared to what baseline, on what class balance, measured on data it never saw, with what cost for each error type? Those four questions *are* the answer.

Q: When ML vs. plain rules? A: If three if-statements solve it, ship the if-statements. ML earns its complexity when patterns are too subtle/numerous for rules and you have labeled data and a way to measure.

Q: Does classical ML still matter in the LLM era? A: More than ever: boosted trees still win tabular problems; and every LLM eval, A/B test, and metrics dashboard is applied statistics. The evals guide (folder 01) is this guide wearing a different hat.

Q: What's the single highest-leverage habit? A: Manual error analysis. Reading your model's mistakes beats another round of hyperparameter tuning almost every time.

8 Final Cheat Sheet (One-Page Revision)

Statistics core: plot first • median+percentiles for skewed data • SD/spread • standard error shrinks with sqrt(n) • CI = plausible range • p-value = surprise under "no effect," NOT probability you're right • significance ≠ importance (effect sizes!) • pre-register metrics, don't peek • correlation ≠ causation (confounders) • Simpson's paradox • regression to the mean • base rates (rare events drown in false positives) • survivorship bias.

Learning mechanics: model = parametrized function • training = minimize loss by gradient descent • learning rate = step size • bias (underfit) vs. variance (overfit) diagnosed by train-vs-validation gap • regularization = penalize complexity (L2 shrinks, L1 zeros, dropout, early stopping, tree depth).

Discipline: train / validation / test — test touched once • k-fold CV • leakage kills: preprocess inside folds, chronological splits for time data, hunt proxy features & duplicates • always beat a baseline.

Algorithm picker: numbers → linear regression first • categories → logistic regression first • tabular & serious → random forest, then XGBoost/LightGBM (the tabular kings) • similarity → kNN (vector search = kNN on embeddings) • segments → k-means • compress/visualize → PCA • raw images/audio/text → neural nets.

Evaluation: regression → MAE/RMSE/R^2 vs. baseline • classification → confusion matrix, precision (alarm quality) vs. recall (catch rate), F1, threshold from business costs, ROC-AUC (AUC-PR when imbalanced), calibration • report uncertainty • slice by segment • read 50 errors by hand.

The one rule: *Any score measured on data the model has seen — or tuned against — is flattery. Generalization, measured honestly, is the entire game.*